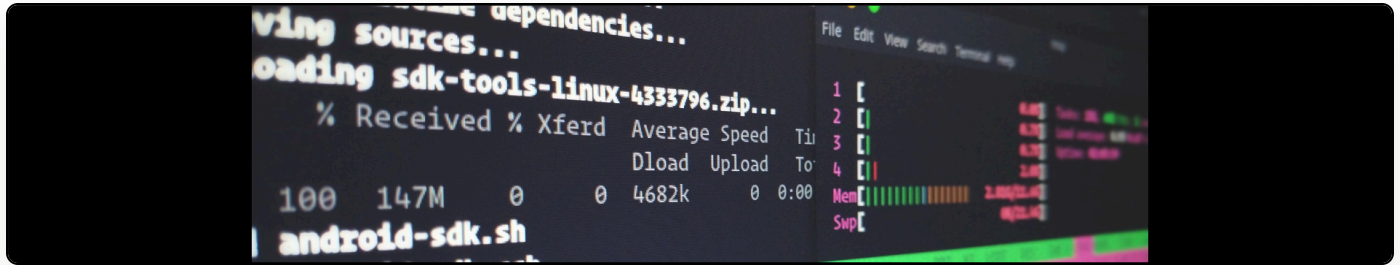


Creating an IRC front/back-end from a C# web application

Cite as: Martin Paul Eve, "Creating an IRC front/back-end from a C# web application", *eve.gd*: Martin Paul Eve, September 17, 2008, <https://doi.org/10.59348/pbszj-40621>.



This lengthy howto will show you how to hook up C# to an eggdrop IRC bot. I've taken this approach because it avoids the overhead of managing a fully fledged IRC client in C# whilst still providing 2-way command functionality between IRC and the application.

The app works by:

1. Connecting to the eggdrop
2. Logging in to the eggdrop
3. Sending .say and .topic commands to the eggdrop
4. Providing a web service to which the eggdrop may send a SOAP request to perform specific actions

So, let's get started!

The first part is the main IRC class. The version implemented here comes with a log4net appender implementation so that errors can be logged directly to IRC.

IRC.cs

```

using System;
using System.Collections.Generic;
using System.Text;
using System.Net;
using System.Net.Sockets;
using System.Collections.Specialized;
using System.Configuration;
using System.IO;
using System.Threading;

namespace Logging
{
    public class IRCLogger : log4net.Appender.AppenderSkeleton
    {
        protected override void Append(log4net.Core.LoggingEvent loggingEvent)
        {
            if (loggingEvent.Level == log4net.Core.Level.Error)
            {
                if (loggingEvent.ExceptionObject != null)
                {
                    IRC.SendAdminMessage(loggingEvent.MessageObject.ToString() + ": " +
loggingEvent.ExceptionObject.Message);
                }
                else
                {
                    IRC.SendAdminMessage(loggingEvent.MessageObject.ToString());
                }

                DateTime gmt = DateTime.Now.AddHours(5);

                IRC.SetAdminTopic("Last error received on " + gmt.ToShortDateString()
+ " at " + gmt.ToShortTimeString() + " (GMT).");
            }
        }
    }

    public class IRC
    {
        static TcpClient client = null;
        static NetworkStream ns = null;
        static StreamWriter sw = null;
        static StreamReader sr = null;
        static bool isConnecting = false;
        static bool isSending = false;

        static List<string> commands = new List<string>();
    }
}

```

```

private static void GetConnection()
{
    bool ret = false;

    if (isConnecting) ret = true;

    while (isConnecting) { Thread.Sleep(1000); }

    if (ret) return;

    isConnecting = true;
    try
    {
        NameValueCollection botConfig =
ConfigurationManager.GetSection("ircSetup/botDetails")
as NameValueCollection;

        client = new TcpClient(botConfig["host"],
int.Parse(botConfig["port"]));

        if (client.Connected)
        {
            ns = client.GetStream();

            sw = new StreamWriter(ns);
            sr = new StreamReader(ns);

            sw.WriteLine(botConfig["username"]);
            sr.ReadLine();
            sw.WriteLine(botConfig["password"]);
            sr.ReadLine();

            sw.Flush();
        }
    }
    finally
    {
        isConnecting = false;
    }
}

public static void SetTopic(string topic)
{
    commands.Add(".topic #default [-http://mysite.com-] " + topic);
    DoSend();
}

public static void SendMessage(string msg)

```

```

{
    commands.Add(".say #default [-default-] " + msg);
    DoSend();
}

public static void SendMessage(string msg, string channel)
{
    commands.Add(".say " + channel + " [-default-] " + msg);
    DoSend();
}

public static void SetAdminTopic(string topic)
{
    commands.Add(".topic #adminchannel [-admin-] " + topic);

    DoSend();
}

public static void SendAdminMessage(string msg)
{
    commands.Add(".say #adminchannel [-admin-] " + msg);

    DoSend();
}

private static void DoSend()
{
    ThreadStart pts = new ThreadStart(SendToBot);
    Thread t = new Thread(pts);
    t.Start();
}

private static void SendToBot()
{
    while (isSending) { System.Threading.Thread.Sleep(1000); }

    isSending = true;

    try
    {
        while (commands.Count > 0)
        {

            //Filter the data using safe whitelist

            string msg = commands[0].ToString();
            string regex = @".*[\r\n].*";

```

```

        if (System.Text.RegularExpressions.Regex.IsMatch(msg, regex))
            return;

        int attempts = 0;

        if (client == null)
        {
            GetConnection();
            attempts++;

            if (attempts > 3) return;
        }

        while (client == null || !client.Connected)
        {
            GetConnection();
            attempts++;

            if (attempts > 3) return;
        }

        if (client.Connected)
        {
            sw.WriteLine(msg);

            sw.Flush();

            sr.ReadLine();
        }

        while (ns.DataAvailable)
        {
            sr.ReadLine();
        }

        commands.RemoveAt(0);
    }
}
catch (Exception ex)
{
    Logging.Logger.Log.Info("Error sending to IRC: " + ex.Message);
}
finally
{
    isSending = false;
}
}

```

```
}  
}
```

Then, put the relevant stuff into your web.config file:

```
<configSections>  
  
  <section name="log4net"  
    type="log4net.Config.Log4NetConfigurationSectionHandler  
    , log4net"  
    requirePermission="false"/>  
  
  <sectionGroup name="ircSetup">  
    <section name="botDetails" type="System.Configuration.NameValueSectionHandler"/>  
  </sectionGroup>  
  
</configSections>  
  
<ircSetup>  
  <botDetails>  
    <add key="host" value="your.eggdrop.host"/>  
    <add key="port" value="31337"/>  
    <add key="username" value="youreggdropusername" />  
    <add key="password" value="youreggdroppassword" />  
  </botDetails>  
</ircSetup>  
  
<log4net>  
  <appender name="ircAppender" type="Logging.IRCLogger, Logging">  
</log4net>
```

Tada - it should hook up! But what about getting commands from the bot I hear you cry? Well, first of all you'll need to hook up the following TCL to your eggdrop:

```

package require http 2.7

bind pubm - * webServiceRequest

proc webServiceRequest {nick uhost hand chan text} {
    if {[string index $text 0] == "!"} {
        set token [::http::geturl "http://www.yoursite.com/webservice.asmx" \
            -type "text/xml; charset=utf-8" \
            -headers {"SOAPAction" "http://www.yoursite.com/IrcMessage"} \
            -query "<?xml version=\\"1.0\\" encoding=\\"utf-8\\"?> \
                <soap:Envelope
xmlns:xsi=\\"http://www.w3.org/2001/XMLSchema-instance\\"
xmlns:xsd=\\"http://www.w3.org/2001/XMLSchema\\"
xmlns:soap=\\"http://schemas.xmlsoap.org/soap/envelope/\\"> \
                    <soap:Header> \
                        <authentication
xmlns=\\"http://www.yoursite.com/\\"> \
                            <user>YourUsername</user> \
                            <password>YourPassword</password> \
                            <version>Bot_1.0</version> \
                        </authentication> \
                    </soap:Header> \
                    <soap:Body> \
                        <ircMessage
xmlns=\\"http://www.yoursite.com/\\"> \
                            <user>$nick</user> \
                            <hostmark>$uhost</hostmark> \
                            <channel>$chan</channel> \
                            <message>$text</message> \
                        </ircMessage> \
                    </soap:Body> \
                </soap:Envelope>"]

        upvar #0 $token state
        if {$state(status) != "ok"} {
            putserv "PRIVMSG $chan :ERROR"
        }
    }
}

```

Then, the webservice at <http://www.yoursite.com/webService.asmx> should expose the following web method:

```
[WebMethod]
[PrincipalPermissionAttribute(SecurityAction.Demand, Role = "Bot")]
[SoapHeader("authentication")]
public void IrcMessage(string user, string hostmark, string channel, string
message)
{
    IRC.ParseIrcMessage(user, hostmark, channel, message);
}
```

With which you can do what you like! The only proviso you might require is the following setup for the authentication SOAP header:

```
/// <summary>
/// An Authentication class
/// </summary>
public class Authentication : SoapHeader
{
    public string User;
    public string Password;
    public string Version;
}
```

And the following HttpModule:

```

using System;
using System.Web;
using System.IO;
using System.Xml;
using System.Xml.XPath;
using System.Text;
using System.Web.Services.Protocols;
using System.Security.Principal;
using System.Web.Security;
using System.Diagnostics;
using Logging;

namespace YourWebService
{

    public sealed class WebServiceAuthenticationModule : IHttpModule
    {

        public void Dispose()
        {
        }

        public void Init(HttpApplication app)
        {
            app.AuthenticateRequest += new
                EventHandler(this.OnEnter);
        }

        public string ModuleName
        {
            get { return "WebServiceAuthentication"; }
        }

        void OnEnter(Object source, EventArgs eventArgs)
        {
            HttpApplication app = (HttpApplication)source;
            HttpContext context = app.Context;
            Stream HttpStream = context.Request.InputStream;

            // Save the current position of stream.
            long posStream = HttpStream.Position;

            // If the request contains an HTTP_SOAPACTION
            // header, look at this message.
            if (context.Request.ServerVariables["HTTP_SOAPACTION"]
                == null)
                return;
        }
    }
}

```

```

// Load the body of the HTTP message
// into an XML document.
XmlDocument dom = new XmlDocument();
string soapUser;
string soapPassword;
string soapVersion;

try
{
    dom.Load(HttpStream);

    // Reset the stream position.
    HttpStream.Position = posStream;

    // Bind to the Authentication header.

    soapUser = dom.GetElementsByTagName("User").Item(0).InnerText;
    soapPassword = dom.GetElementsByTagName("Password").Item(0).InnerText;
    soapVersion = dom.GetElementsByTagName("Version").Item(0).InnerText;

    if (Membership.ValidateUser(soapUser, soapPassword))
    {
        MembershipUser user = Membership.GetUser(soapUser);
        GenericIdentity identity = new GenericIdentity(user.UserName);
        RolePrincipal rp = new RolePrincipal(identity);

        HttpContext.Current.User = rp;
        user.Comment = soapVersion;

        Membership.UpdateUser(user);

        Logger.Log.Info("Authenticated: " + soapUser + " :: " +
soapVersion);
    }
    else
    {
        Logger.Log.Info("Failed to login as " + soapUser);
        Logging.IRC.SendAdminMessage("Attempt to login as " + soapUser + "
with password " + soapPassword + " failed.");
    }
}
catch (Exception e)
{
    Logging.IRC.SendAdminMessage("Error in SOAP parser: " + e.Message);

    // Reset the position of stream.
    HttpStream.Position = posStream;
}

```

```
        // Throw a SOAP exception.  
        XmlQualifiedName name = new  
            XmlQualifiedName("Load");  
        SoapException soapException = new SoapException(  
            "Unable to read SOAP request", name, e);  
        throw soapException;  
    }  
    return;  
}  
}  
}
```

Which, of course, you will need to load in web.config:

```
<httpModules>  
    <add name="WebServiceAuthenticationModule"  
type="YourWebService.WebServiceAuthenticationModule" />  
</httpModules>
```